

```
def get_thyroid_test_input():
    """Get thyroid test results from user input."""
    print("\nEnter thyroid test results (up/down/normal/not checked):")
    tests = {
        'TSH': input("TSH: ").lower(),
        'T3': input("T3: ").lower(),
        'free_T4': input("free T4: ").lower(),
        'PRL': input("PRL: ").lower(),
        'Na': input("Na: ").lower(),
        'TG': input("TG: ").lower(),
        'Hb': input("Hb: ").lower(),
        'chol': input("chol: ").lower(),
        'anti_TPO': input("anti-TPO: ").lower(),
        'TSH_second_time': input("TSH second test: ").lower(),
        'T4_second_time': input("T4 second test: ").lower(),
        'T3_second_time': input("T3 second test: ").lower()
    }
    return tests
```

```
def get_symptoms_input():
    """Get symptom information from user input."""
    print("\nEnter symptom information (yes/no):")
    symptoms = {
        'fatigue': input("Fatigue: ").lower(),
        'constipation': input("Constipation: ").lower(),
        'feeling_cold': input("Feeling cold: ").lower(),
        'goiter': input("Goiter: ").lower(),
        'weight_gain': input("Weight gain: ").lower(),
        'dry_skin': input("Dry skin: ").lower(),
        'hair_loss': input("Hair loss: ").lower(),
        'depression': input("Depression: ").lower(),
        'memory_problems': input("Memory problems: ").lower(),
        'muscle_weakness': input("Muscle weakness: ").lower()
    }
    return symptoms
```

```
def print_management_notes():
    """Print standard management notes for hypothyroidism."""
    notes = """
Management Notes:
1. Visit for TSH test if you have any hypo or hyper symptoms
2. Visit for TSH test if you become pregnant
3. Visit for TSH test if you start menopause or HRT/OCP treatment
4. Visit for TSH test if you start medications that interact with levothyroxine
5. Visit for TSH test if you gain or lose 10% of your weight
```

6. Visit for TSH test if you change your levothyroxine brand

Levothyroxine Usage Instructions:

- Take on an empty stomach
- Wait 30 minutes before eating
- Take with a glass of water
- Avoid other medications/supplements for 4 hours
- No interaction with diabetes or blood pressure medications

"""

print(notes)

def initial_screening():

"""First visit for patients with suspicious symptoms."""

print("\n=== Initial Screening Visit ===")

symptoms = get_symptoms_input()

if any(value == 'yes' for value in symptoms.values()):

print("\nRecommendation: Check TSH and free T4 levels")

else:

risk_factors = {

'goiter': input("Does patient have goiter? (yes/no): ").lower(),

'autoimmune_disease': input("Any autoimmune disease? (yes/no): ").lower(),

'diabetes': input("Diabetes? (yes/no): ").lower(),

'family_history': input("Family history of thyroid disease? (yes/no): ").lower(),

'pericardial_effusion': input("Pericardial effusion? (yes/no): ").lower()

}

if any(value == 'yes' for value in risk_factors.values()):

print("\nRecommendation: Check TSH level")

else:

print("\nNo thyroid testing recommended at this time")

def confirm_diagnosis():

"""Confirm diagnosis of hypothyroidism."""

print("\n=== Diagnosis Confirmation Visit ===")

tests = get_thyroid_test_input()

if tests['TSH'] == 'normal':

symptoms = get_symptoms_input()

if any(value == 'yes' for value in symptoms.values()):

print("\nRecommendation: Repeat TSH and free T4 testing")

if tests['T4_second_time'] == 'down':

status = "central hypothyroidism"

print(f"\nDiagnosis: {status}")

else:

```

print("\nHypothyroidism ruled out")

elif tests['TSH'] == 'up':
    if tests['free_T4'] == 'down':
        status = "overt hypothyroidism"
        print(f"\nDiagnosis: {status}")
    elif tests['T3'] == 'normal' and tests['free_T4'] == 'normal':
        status = "subclinical hypothyroidism"
        print(f"\nDiagnosis: {status}")
        print("Recommendation: Check anti-TPO, repeat TSH and free T4 in 1-3 months")
        print("If patient is pregnant, check TSH and free T4 next week")

if (tests['TSH'] == 'up' and tests['T3'] == 'normal' and tests['free_T4'] == 'normal' and
    tests['T3_second_time'] == 'normal' and tests['TSH_second_time'] == 'normal' and
    tests['T4_second_time'] == 'normal' and tests['anti_TPO'] == 'positive'):
    print("\nRecommendation: Follow up with TSH and free T4 annually")

if 'goiter' == 'yes' and tests['TSH'] == 'normal' and tests['free_T4'] == 'normal':
    print("\nRecommendation: Check anti-TPO antibodies")

return status if 'status' in locals() else None

def management_plan(status=None):
    """Create management plan for confirmed hypothyroidism."""
    if not status:
        status = input("\nEnter patient status (central hypo/overt hypo/subclinical hypo): ").lower()

    print(f"\n=== Management Plan for {status} ===")

    age = int(input("Patient age: "))
    weight = float(input("Patient weight (kg): "))
    TSH = float(input("Current TSH level: "))
    upper_limit_normal_TSH = float(input("Lab's upper limit for normal TSH: "))
    ACD = input("Acute coronary disease? (yes/no): ").lower() == 'yes'
    IHD = input("Ischemic heart disease? (yes/no): ").lower() == 'yes'

    symptoms = get_symptoms_input()
    symptomatic = any(value == 'yes' for value in symptoms.values())

    if status == "subclinical hypothyroidism":
        if TSH > 10:
            if age > 65:
                print("\nRecommendation: Levothyroxine 25 mcg daily, recheck TSH in 6 weeks")
            else:

```

```

print("\nRecommendation: Levothyroxine 50 mcg daily, recheck TSH in 6 weeks")

elif 7 < TSH <= 10:
    if age > 65:
        if symptomatic:
            print("\nRecommendation: Levothyroxine 25 mcg daily, recheck TSH in 6 weeks")
        else:
            print("\nRecommendation: No treatment needed, follow up TSH and free T4 in 6
months")
    else:
        print("\nRecommendation: Levothyroxine 50 mcg daily, recheck TSH in 6 weeks")

elif upper_limit_normal_TSH < TSH <= 7:
    if age > 65:
        print("\nRecommendation: No treatment needed, follow up TSH and free T4 in 6
months")
    else:
        print("\nRecommendation: Levothyroxine 50 mcg daily, recheck TSH in 6 weeks")

elif status == "overt hypothyroidism":
    levo_dose = min(1.6 * weight, 200) # mcg/day

    if age > 65 or ACD or IHD:
        duration = input("Less than 2 months since hypothyroidism onset? (yes/no): ").lower()
        if duration == 'yes':
            levo_dose = 1.2 * weight
        else:
            levo_dose = 25 if age > 70 else 50

    print(f"\nRecommendation: Levothyroxine {levo_dose:.0f} mcg daily, recheck TSH in 6
weeks")

print_management_notes()

def follow_up_visit():
    """Follow-up visit for patients on levothyroxine."""
    print("\n=== Follow-up Visit ===")

    age = int(input("Patient age: "))
    status = input("Patient status (central hypo/overt hypo/subclinical hypo): ").lower()
    TSH = float(input("Current TSH level: "))
    free_T4 = float(input("Current free T4 level: "))
    lower_limit_free_T4 = float(input("Lab's lower limit for free T4: "))
    upper_limit_free_T4 = float(input("Lab's upper limit for free T4: "))

```

```

# Determine target TSH range
if age < 65:
    target_TSH_min, target_TSH_max = 0.5, 2.0
else:
    target_TSH_min, target_TSH_max = 3.0, 5.0

# Assess current TSH level
if TSH > target_TSH_max:
    TSH_status = "up"
elif target_TSH_min <= TSH <= target_TSH_max:
    TSH_status = "normal"
else:
    TSH_status = "low"

print(f"\nTarget TSH range for this patient: {target_TSH_min}-{target_TSH_max}")

if status == "subclinical hypothyroidism":
    compliance = input("Has patient been taking medication regularly? (yes/no): ").lower()
    interactions = input("Any medications taken within 4 hours of levothyroxine? (yes/no): ").lower()

    if compliance == 'no' or interactions == 'yes':
        print("\nRecommendation:")
        print("- Take medication regularly on empty stomach")
        print("- No other medications/supplements within 4 hours")
        print("- Recheck TSH in 6 weeks")

    if TSH_status == "normal":
        print("\nRecommendation: Continue current dose, follow up TSH annually (or every 6 months if compliance concerns)")
    elif TSH_status == "up":
        print("\nRecommendation: Increase dose by 12.5-25 mcg daily, recheck TSH in 6 weeks")

elif status == "overt hypothyroidism":
    if TSH_status == "normal":
        print("\nRecommendation: Continue current dose, follow up TSH annually")
    elif TSH > target_TSH_max:
        if TSH <= 10:
            print("\nRecommendation: Increase dose by 12.5-25 mcg daily (or additional 50/100 mcg on Fridays)")
        elif 10 < TSH <= 20:
            print("\nRecommendation: Increase dose by 25-50 mcg daily (or additional 50/100 mcg on Thursdays and Fridays)")
        elif TSH > 20:

```

```

    if free_T4 < lower_limit_free_T4:
        print("\nRecommendation: Double current dose, recheck TSH in 6 weeks")
    elif free_T4 > upper_limit_free_T4:
        print("\nRecommendation: Add one extra dose per week (e.g., 2 pearls on
Fridays)")
    elif TSH_status == "low":
        if 0.05 < TSH < target_TSH_min:
            print("\nRecommendation: Reduce dose by one dose per week (e.g., skip Fridays)")
        elif TSH < 0.05:
            print("\nRecommendation: Reduce dose by 50%, recheck TSH in 6 weeks")

print_management_notes()

def main():
    """Main function to run the hypothyroidism assessment tool."""
    print("\n=== Hypothyroidism Assessment and Management Tool ===")

    while True:
        print("\nSelect visit type:")
        print("1. Initial screening (suspicious symptoms)")
        print("2. Confirm diagnosis")
        print("3. Management plan for new diagnosis")
        print("4. Follow-up visit on treatment")
        print("5. Exit")

        choice = input("\nEnter your choice (1-5): ")

        if choice == '1':
            initial_screening()
        elif choice == '2':
            confirm_diagnosis()
        elif choice == '3':
            management_plan()
        elif choice == '4':
            follow_up_visit()
        elif choice == '5':
            print("\nExiting program...")
            break
        else:
            print("\nInvalid choice. Please enter a number between 1-5.")

if __name__ == "__main__":
    main()

```